

Lab. 11
Stored Routines, Triggers & NoSQL

Content

1. Logic that lives in the database
2. Stored procedures: IN and OUT parameters
3. Stored functions
4. Triggers: actions that run automatically
5. Calling routines from Node.js; running the starter (npm run dev)
6. MySQL's NoSQL side: the JSON data type
7. The document store with the X DevAPI
8. Hands-on exercises; good habits; next: Lab 12
9. Summary

Recap

- Lab 9: a transaction makes several statements commit together or roll back together.
- Lab 10: indexes and EXPLAIN keep queries fast; ? placeholders and least privilege keep them safe.
- So far, every rule of your app (take a seat, count courses) lives in your Node.js code.
- Today: logic that lives inside MySQL itself, and MySQL's NoSQL side (JSON and documents).

Logic that Lives in the Database

Stored Routines

- A stored routine is code saved inside MySQL and run by the server when asked.
- Reuse: the same logic can be called from your API, from Workbench or from any other client.
- Fewer round-trips: a multi-step job runs next to the data in one call.
- **Three kinds:** procedures (do something), functions (return a value), triggers (run by themselves).

Stored Procedures: IN and OUT Parameters

A Procedure: a named block of SQL you run with CALL. IN parameters pass values in.

```
DELIMITER //
CREATE PROCEDURE enrol_student(IN p_student BIGINT, IN p_course INT)
BEGIN
  INSERT INTO enrolments (student_id, course_id) VALUES (p_student, p_course);
END //
DELIMITER ;

CALL enrol_student(3, 1);    -- run it: student 3 joins course 1
```

DELIMITER // switches the end-of-statement marker, so the ; inside BEGIN ... END is not read as the end of CREATE PROCEDURE. Switch it back with DELIMITER ;

Stored Procedures: IN and OUT Parameters (Contd.)

OUT Parameters: an OUT parameter hands a result back to the caller.

```
DELIMITER //
CREATE PROCEDURE count_students(IN p_course INT, OUT p_total INT)
BEGIN
  SELECT COUNT(*) INTO p_total FROM enrolments WHERE course_id = p_course;
END //
DELIMITER ;

CALL count_students(1, @n);
SELECT @n;            -- read the OUT value
```

SELECT ... INTO stores a query result in a variable. @n is a session variable that receives the OUT value.

Stored Functions

A Function RETURNS one value, so you can use it inside SELECT, WHERE or ORDER BY, like a built-in.

```
DELIMITER //
CREATE FUNCTION course_count(p_student BIGINT) RETURNS INT
READS SQL DATA
BEGIN
  DECLARE n INT;
  SELECT COUNT(*) INTO n FROM enrolments WHERE student_id = p_student;
  RETURN n;
END //
DELIMITER ;

SELECT Id, name, course_count(Id) AS courses FROM students;
```

READS SQL DATA says the function only reads data. Leave it out and a default MySQL 8 install refuses with Error 1418.

Stored Functions (Contd.)

Lab Exercise 1: *A procedure and a function*

- ❑ Open sql/exercise1.sql in Workbench (after running sql/lab11_setup.sql once).
- ❑ Write enrol_student(p_student, p_course) that inserts an enrolment, and CALL it.
- ❑ Write course_count(p_student) and use it in a SELECT listing every student.
- ❑ Compare with a plain GROUP BY query, then try an OUT parameter: count_students.

Starter code:

```
CREATE PROCEDURE enrol_student(IN p_student BIGINT, IN p_course INT)
BEGIN
  -- TODO: INSERT INTO enrolments (student_id, course_id) VALUES (...);
END //
```

Triggers: Actions that Run Automatically

- A trigger is code that runs by itself whenever a row is inserted, updated or deleted.
- It fires BEFORE or AFTER the change, once FOR EACH ROW.
- Inside it, NEW is the new row (INSERT, UPDATE) and OLD is the old row (UPDATE, DELETE).
- Great for keeping data in step automatically, such as free seats, or for an audit log.
- The application never asks for it, so a trigger is hidden: document every one.

Triggers: Actions that Run Automatically (Contd.)

A Trigger Example: every new enrolment takes a seat, with the guard from Lab 9.

```
DELIMITER //
CREATE TRIGGER after_enrolment_insert
AFTER INSERT ON enrolments
FOR EACH ROW
BEGIN
  UPDATE courses SET seats = seats - 1
   WHERE id = NEW.course_id AND seats > 0;
  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'course is full';
  END IF;
END //
DELIMITER ;
```

NEW.course_id is the course of the row just inserted. If no seat was free, SIGNAL raises an error and the INSERT is undone too.

Triggers: Actions that Run Automatically (Contd.)

Giving the Seat Back, and Why No Transaction Is Needed

- A DELETE trigger reads **OLD**, the row that was removed.
- An INSERT and its trigger are **one statement**: they succeed or fail together.
- SIGNAL SQLSTATE '45000' cancels the whole statement; Node receives the error message.

```
CREATE TRIGGER after_enrolment_delete
AFTER DELETE ON enrolments
FOR EACH ROW
  UPDATE courses SET seats = seats + 1 WHERE id = OLD.course_id;
```

In Lab 9, Node wrapped two statements in a transaction. Here MySQL does both steps inside one statement.

Triggers: Actions that Run Automatically (Contd.)

Lab Exercise 2: *Triggers keep the seats in step*

- ❑ Open sql/exercise2.sql. Re-sync the seats (Step 1 is done for you).
- ❑ Create after_enrolment_insert: take a seat, and SIGNAL when the course is full.
- ❑ Create after_enrolment_delete: give the seat back. Test both with INSERT / DELETE.
- ❑ Enrol from the web page: the seats drop by 2, not 1! Write down why (Exercise 3 fixes it).

Starter code:

```
CREATE TRIGGER after_enrolment_insert AFTER INSERT ON enrolments
FOR EACH ROW
BEGIN
  -- TODO: guarded UPDATE of courses.seats, then IF ROW_COUNT() = 0 THEN SIGNAL ...
END //
```

Calling Routines from Node.js

From mysql2: a procedure is just another statement; a function works inside any SELECT.

```
// POST /students/:id/enrolments  - one statement, MySQL does the rest
await pool.execute('CALL enrol_student(?, ?)', [studentId, courseId]);
res.status(201).json({ ok: true });

// ...and in the catch block:
if (err.sqlState === '45000')              // our SIGNAL
  return res.status(409).json({ error: err.sqlMessage });  // course is full

// GET /students  - the function inside a SELECT
const [rows] = await pool.execute(
  'SELECT Id AS id, name, major, course_count(Id) AS courses FROM students');
```

Keep using ? placeholders when you CALL a procedure (Lab 10: never glue values into SQL).

Calling Routines from Node.js (Contd.)

Watch Out: One Seat, Taken Twice

- The starter still has your Lab 9 enrol: Node runs UPDATE seats ... and then the INSERT.
- Once the trigger exists, the INSERT takes a seat as well: one enrolment costs two seats.
- The web page shows it in the log: **seats 28 → 26 (−2), expected −1**.
- Fix: let **one** place own the rule. Node calls the procedure; the trigger moves the seats.
- This is exactly why a hidden trigger must be documented for the whole team.

Running the Starter Project

From the project folder: open a terminal in Lab11_Exercise_Starter (VS Code: Terminal → New Terminal) and run:

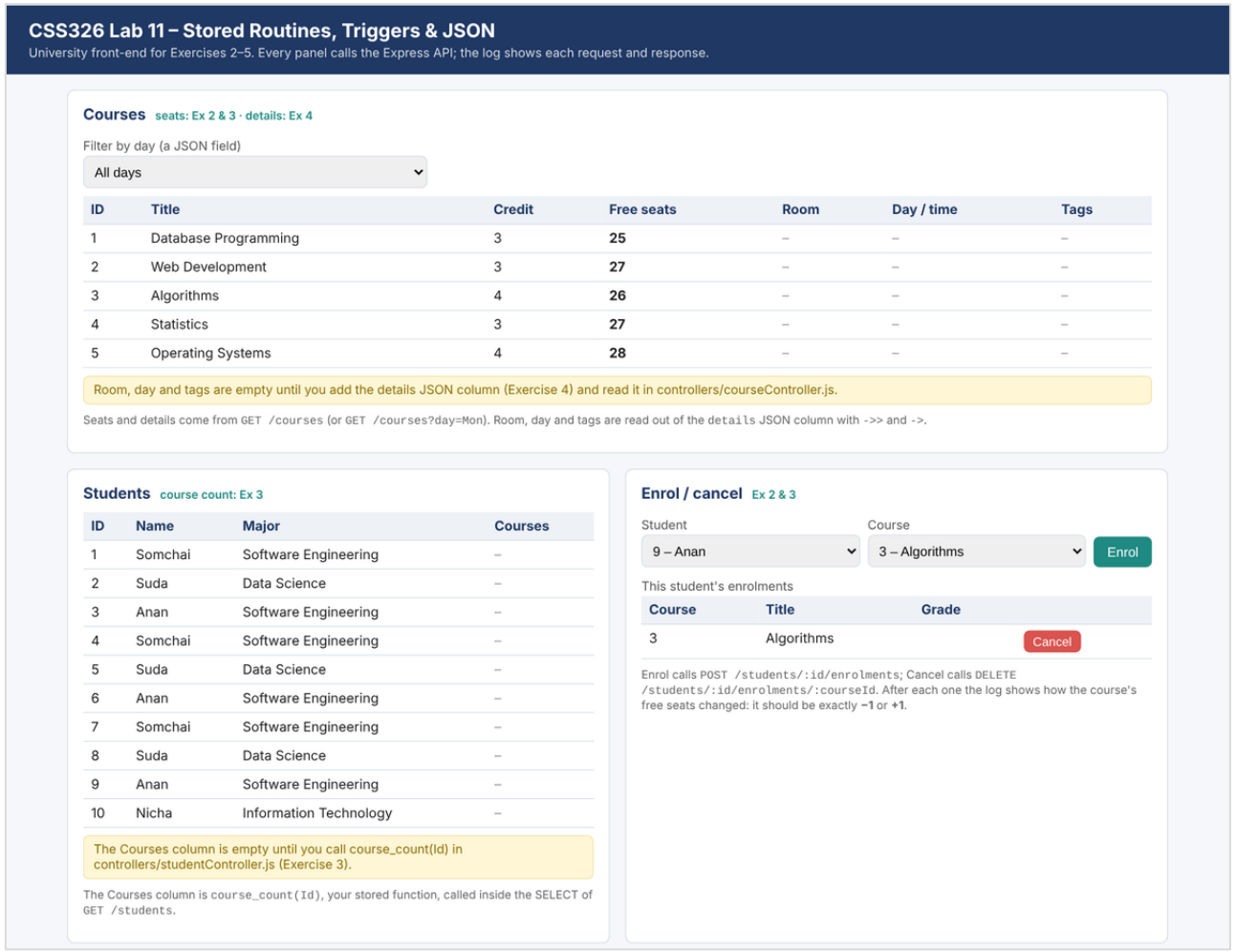
npm install	← once: express, mysql2, dotenv, @mysql/xdevapi
npm run dev	← starts the server (runs node --watch server.js)
	Lab 11 server on http://localhost:3000
Ctrl + C	← stops the server

Before you start: *Run sql/lab11_setup.sql in Workbench first, then put your MySQL login in the .env file (DB_USER, DB_PASS), as in Lab 10. db.js and the document store both read it. Open http://localhost:3000 for the front-end.*

Running the Starter Project (Contd.)

The Lab 11 Web App

- Courses: free seats, plus room, day and tags from JSON (Ex 4).
- Students: the Courses column is your function (Ex 3).
- Enrol / Cancel: the procedure and the triggers (Ex 2, 3).
- Reviews: the document store (Ex 5).
- Yellow notes say which exercise fills each part; the log shows every response and seat change.



Running the Starter Project (Contd.)

If something fails, read the first error line:

- **Access denied for user 'root'** → set your own DB_USER / DB_PASS in .env.
- **Access denied for user ''@'localhost'** → .env is missing, misnamed (.env.txt) or not saved.
- **Unknown database 'lab11'** → run sql/lab11_setup.sql in Workbench first.
- **FUNCTION lab11.course_count does not exist** → finish Exercise 1 first.
- **Error 1418 … DETERMINISTIC, NO SQL, or READS SQL DATA** → add READS SQL DATA to the function.
- **Error 1644: course is full** → not a bug: your trigger SIGNALled. Answer 409 in Node.
- **ECONNREFUSED …:33060** → the X Protocol is off; Exercise 5 is optional, skip it.

Calling Routines from Node.js (Contd.)

Lab Exercise 3: *Use the routines from the web app*

- In controllers/enrolmentController.js, replace the Lab 9 transaction in enrol with CALL enrol_student.
- Map err.sqlState '45000' to 409; keep 409 for duplicates and 404 for unknown students.
- In unenrol keep only the DELETE: the trigger gives the seat back.
- In studentController.js add course_count(Id) AS courses. Each enrol/cancel must now show −1 / +1.

Starter code:

```
await pool.execute('CALL enrol_student(?, ?)', [studentId, courseId]);
res.status(201).json({ ok: true, student_id: studentId, course_id: courseId });
```

Procedures vs Application Logic

- Routines keep a rule next to the data, and every client gets the same rule.
- But database code is harder to version, test and debug than code in your app.
- Many web apps keep most logic in Node and use routines or triggers for a few data-heavy jobs.
- Pick one home per rule. Never enforce the same rule twice: in Node and in a trigger.

Procedures vs Application Logic (Contd.)

Procedure, Function or Trigger?

	Stored procedure	Stored function	Trigger
How it runs	You CALL it	Inside a query: SELECT, WHERE, ORDER BY	By itself, on INSERT / UPDATE / DELETE
Gives back	OUT parameters or result rows	Exactly one value (RETURNS)	Nothing: it reacts to the change
Changes data?	Yes: that is its main job	Should not (READS SQL DATA)	Yes: keeps other rows in step
Own transaction?	Yes: START TRANSACTION … COMMIT	No	No: it is part of the statement that fired it
From Node.js	`pool.execute('CALL enrol_student(?, ?)')`	`SELECT …, course_count(Id) …`	Never called: fires on your INSERT
In this lab	enrol_student, count_students	course_count	after_enrolment_insert / _delete

Procedures vs Application Logic (Contd.)

Where Should the Seat Rule Live? The Same Enrolment, Three Ways

1. Lab 9: a transaction in Node

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  const [u] = await conn.execute(
    'UPDATE courses SET seats = seats - 1
     WHERE id = ? AND seats > 0', [c]);
  if (u.affectedRows === 0) throw ...;
  await conn.execute(
    'INSERT INTO enrolments ...', [s, c]);
  await conn.commit();
} catch (e) { await conn.rollback(); }
finally { conn.release(); }
```

2. A procedure that owns the rule

```
CREATE PROCEDURE enrol_student_tx(...)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  UPDATE courses SET seats = seats - 1
   WHERE id = p_course AND seats > 0;
  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000' ...;
  END IF;
  INSERT INTO enrolments ...;
  COMMIT;
END
```

3. Today: procedure + trigger

```
-- procedure: only the INSERT
INSERT INTO enrolments
  (student_id, course_id)
  VALUES (p_student, p_course);

-- trigger: AFTER INSERT ... FOR EACH ROW
UPDATE courses SET seats = seats - 1
 WHERE id = NEW.course_id AND seats > 0;
IF ROW_COUNT() = 0 THEN
  SIGNAL SQLSTATE '45000' ...;
END IF;
```

All three are all-or-nothing. Choose ONE home for the rule: enrol_student plus a seat UPDATE plus the trigger would take the seat twice (Exercise 2).

Procedures vs Application Logic (Contd.)

Which One, When?

- **Transaction in Node:** the rule sits in your app code, easy to test and version; but every other client (another app, Workbench) must repeat it.
- **Procedure with a transaction:** one CALL does both steps on the server, and every client that CALLs it gets the same rule; but a plain INSERT still skips it.
- **Trigger:** nobody can skip it, not even an INSERT typed in Workbench; but it is hidden from anyone who reads only the app code.
- **Atomic in all three:** the UPDATE and the INSERT succeed or fail together (the trigger because it is part of the INSERT statement).
- **This lab:** the procedure is the entry point and the trigger owns the seats. That is why enrol_student does not touch courses: if it did, every enrolment would cost two seats.

MySQL's NoSQL Side: the JSON Data Type

- Not all data fits neat columns: sometimes the fields differ from row to row.
- MySQL has a JSON column type: one column holds a whole flexible object.
- Relational tables and document-style data live side by side in the same database.
- Use it for variable attributes, settings or metadata that don’t deserve their own columns.

MySQL's NoSQL Side: the JSON Data Type (Contd.)

Storing JSON: declare a JSON column, then store an object with JSON OBJECT or a JSON literal.

```
ALTER TABLE courses ADD COLUMN details JSON;

UPDATE courses SET details = JSON_OBJECT('room','BKD 3201', 'day','Mon',
      'time','09:00', 'online', false, 'tags', JSON_ARRAY('core','programming'))
 WHERE id = 1;

-- or a JSON literal; this one has an extra field, zoom
UPDATE courses SET details = '{"room": "Online", "day": "Wed", "zoom": "css-stat"}'
 WHERE id = 4;
```

Rows don't need the same fields: course 4 has zoom, the others don't. MySQL checks that the value is valid JSON.

MySQL's NoSQL Side: the JSON Data Type (Contd.)

Querying JSON: pull fields out with ->> (text) and -> (JSON value), and filter on them.

```
SELECT title, details->>'$.room' AS room, details->>'$.day' AS day
  FROM courses;

SELECT title FROM courses WHERE details->>'$.day' = 'Mon';

SELECT title, details->'$.tags' AS tags FROM courses;   -- a JSON array

// in Node: the value from the page still goes in as a ? parameter
sql += " WHERE details->>'$.day' = ?";  params.push(req.query.day);
```

'$.day' is a path into the document. The long form of -> is JSON_EXTRACT(details, '$.day').

MySQL's NoSQL Side: the JSON Data Type (Contd.)

When to Use JSON

- Columns first: structured data with fixed fields belongs in proper columns.
- JSON for the genuinely variable or sparse attributes that differ across rows.
- JSON fields are harder to index, constrain and join than ordinary columns.
- A good pattern: core fields as columns, one JSON column for the long tail of extras.

MySQL's NoSQL Side: the JSON Data Type (Contd.)

Lab Exercise 4: *JSON in practice*

- ❑ In sql/exercise4.sql, add the details column and store an object for every course.
- ❑ Query each title with its room and day, then only the Monday courses.
- ❑ In controllers/courseController.js return room, day, time and tags with ->> and ->.
- ❑ Support GET /courses?day=Mon with a ? parameter. Try the day filter in the page.

Starter code:

```
SELECT id, title, credit, seats,
       details->>'$.room' AS room, details->>'$.day' AS day, details->'$.tags' AS tags
  FROM courses   -- + WHERE details->>'$.day' = ?  when req.query.day is set
```

The Document Store with the X DevAPI

MySQL as a Document Database: JSON documents in a collection, no CREATE TABLE, no SQL.

```
const mysqlx = require('@mysql/xdevapi');      // X Protocol, port 33060
const session = await mysqlx.getSession({ host: process.env.DB_HOST,
  port: 33060, user: process.env.DB_USER, password: process.env.DB_PASS });
const reviews = await session.getSchema('lab11')
  .createCollection('course_reviews', { reuseExisting: true });

await reviews.add({ course_id: 1, student: 'Ploy', stars: 5, tags: ['labs'] }).execute();

const result = await reviews.find('course_id = :c').bind('c', 1).execute();
res.json(result.fetchAll());
await session.close();
```

add() and find() work on schema-free documents: two reviews may carry different fields. Same server, used as a document store.

The Document Store with the X DevAPI (Contd.)

Port 33060: the X Protocol

- The X DevAPI talks to MySQL through the **X Protocol** on port **33060**. Workbench, mysql2 and the rest of the app use **3306**.
- Same server, same login, just a second door. In the app it comes from DB_X_PORT=33060 in .env (reviewController.js).
- It is built into MySQL 8 and switched on by default. Check it in Workbench:

```
SHOW VARIABLES LIKE 'mysqlx_port';   -- expect 33060
SHOW VARIABLES LIKE 'mysqlx';        -- expect ON
```

ECONNREFUSED 127.0.0.1:33060? *Windows installer: "Enable X Protocol" was unticked (re-run the installer, Reconfigure). Firewall blocks 33060 (only for a remote server). A custom mysqlx_port: put that value in DB_X_PORT. Exercise 5 is optional, so you can skip it.*

The Document Store with the X DevAPI (Contd.)

Lab Exercise 5: *Course reviews in a collection (if time)*

- The login comes from .env; the X Protocol uses DB_X_PORT=33060, not 3306.
- Implement list: find the reviews of one course with find(...).bind(...).
- Implement add: store { course_id, student, stars, comment } and tags only when given.
- Add two reviews in the page, then note one case where a document store fits better than a table.

Starter code:

```
const docs = await withCollection(async (reviews) => {
  const result = await reviews.find('course_id = :c').bind('c', courseId).execute();
  return result.fetchAll();
});
```

Good Habits

- Use routines for shared, data-heavy rules; keep most app logic in Node, where it is easy to test.
- Give each rule one home: if a trigger moves the seats, Node must not move them too.
- Document every trigger, and add READS SQL DATA (or DETERMINISTIC) to functions.
- Columns for structured data, JSON for the variable parts, and keep using ? for every value.

Where we're going next

- You have now seen MySQL as relational, programmable and document-style.
- That completes the technical labs: you have every tool your course project needs.
- Lab 12 is the Final Project Presentation: bring it all together and show what you built.

Summary

- Procedures (CALL, IN / OUT) and functions (RETURNS one value) keep reusable logic in MySQL.
- Triggers run by themselves on INSERT / UPDATE / DELETE, using NEW and OLD; SIGNAL refuses a change.
- From Node: pool.execute('CALL ...', [?]); functions work inside any SELECT; map 45000 to 409.
- JSON columns (JSON_OBJECT, ->>, ->) and the X DevAPI document store give MySQL a NoSQL side.
- Complete Exercises 1–4 in class (5 if time), using the Lab 11 web app.

Thank you & Assignment time…